MISO
Maestría en Ingeniería de Software

Arquitectura detallada

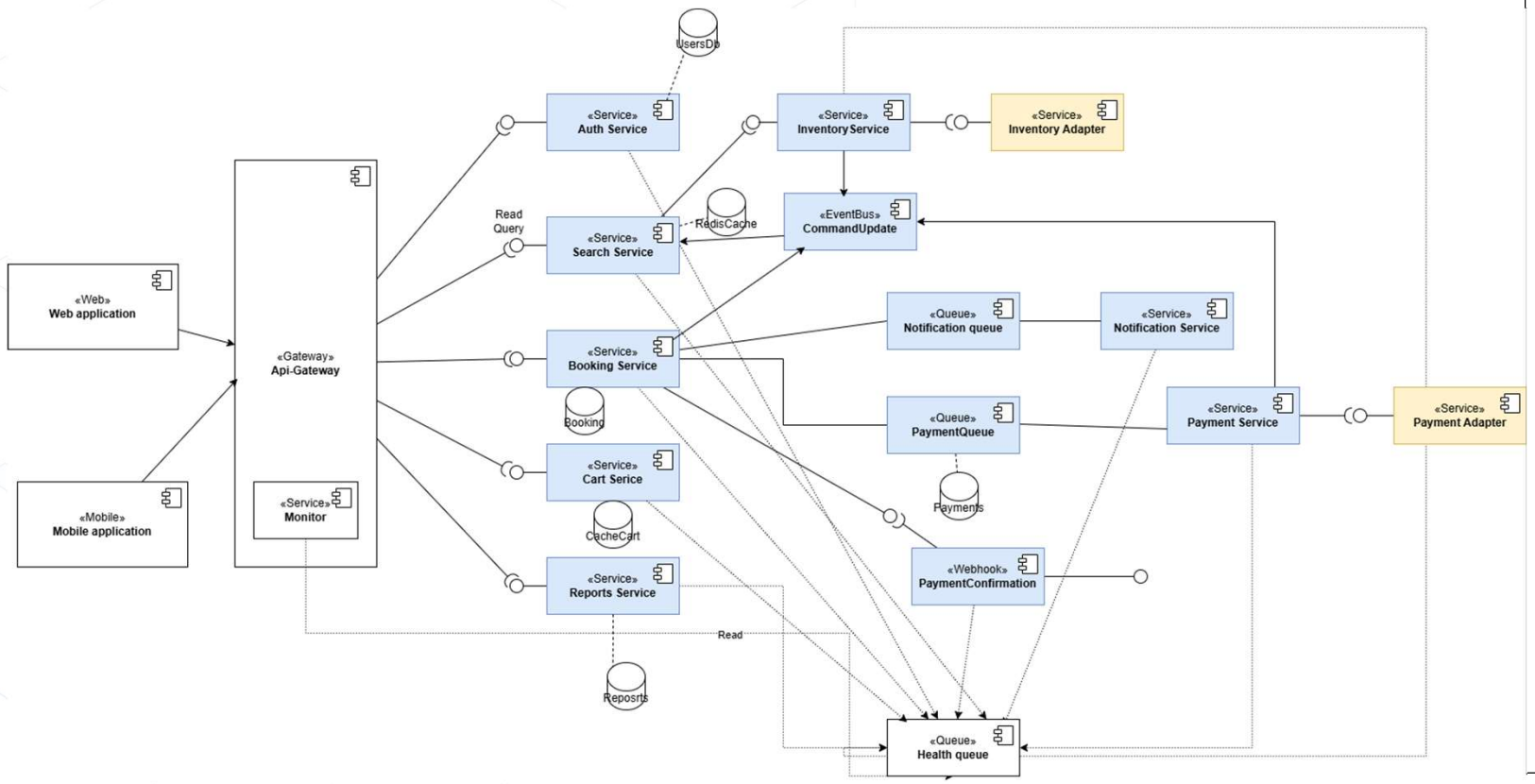
Grupo 11

MISO
Maestría en Ingeniería de Software

Vista Funcional – (Estilo Microservicios)

Proyecto		ID	VC-001	Elaboración	Ronald Carrascal	Versión	1.0
Vista			Modelo	Funcional			

Convención



Vista Funcional – (Estilo Microservicios)

Proyecto		**ID**	VC-001	**Elaboración**	Ronald Carrascal	**Versión**	1.0
Vista			**Modelo**	Funcional			

Tácticas de arquitectura utilizadas para favorecer la disponibilidad y resiliencia

Esta arquitectura nos muestra un estilo de arquitectura basado en microservicios, adicional a esto algunas llamadas son de manera asincrona, utilizando una cola de mensajeria para reealizar tareas que no necesitan de respuesta sincrónica, como lo son la generacion de pagos y el envio de notificaciones.
Se utilizará un servicio de monitoreo que se encargará de recibir heartbeat a travez de una cola de mensajeria, a dicha cola estan reportándose todos los microservicios de manera asincrona, de tal manera que si un servicio deja de enviar la pulsacion, el servicio de monitoreo detectará una posible falla en el servicio para su revision.

El servicio de monitoreo intentara recuperar la falla automaticamente de ser posible, tratando de levantar la instancia (Solo en caso de que la instancia tenga problemas de disponibilidad).
Se usará un servicio de encripcacion de mensajeria al momento de generar los pagos, de tal manera que la informacion viaje cifrada para garantizar que la informacion del pago este segura, asi mismo cada pago tendra un hash unico de tal manera que se peuda verificar si el pago fue exitooso o no.

Razonamiento sobre las principales decisiones de arquitectura tomadas en este modelo

La disponibilidad es uno de los atributos de calidad que deseamos favorecer en esta arquitectura, para ello se utilizaran tácticas de arquitectura como el uso de heartbeat para verificar el estado de salud de los servicios, este hearbeat sera emitido por los servicios a una cola de estado de salud, para que el servicio de monitoreo lea de dicha cola y verifique el estado del microservicio, cabe anotar que el servicio de monitoreo mantendra el estado de cada aplicación, con el fin de detectar la posible falla si un servicio no se reporta.

Una de las principales razones del uso de estas tacticas de arquitectura en mensajeria asincrona es que estas nos permitirán enmascarar posibles fallos que se presenten en algunos servicios, por ejemplo, si tenemos una falla en el servicio de pagos este dejara de reportarse al servicio de monitoreo, lo cual nos indicara de su posible falla para una posible recuperacion automatica(Levantando la instancia), de manera adicional el servicio de pagos recibe mensajes por medio de una cola, de tal manera que si este falla, el mensaje se mantendra encolado hasta que este se recupere, enmascarando asi la falla al cliente, ya que su envio se realizara en cuanto el servicio esté disponible.

Vista Funcional – (Estilo Microservicios)

Proyecto		ID	VC-001	Elaboración	Ronald Carrascal	Versión	1.0
Vista			Modelo	Funcional			

Tácticas de arquitectura utilizadas para favorecer la seguridad

Se utilizará Autenticación y Autorización como táctica de arquitectura para favorecer la seguridad para asegurar que cualquier petición que llegue por medio del api Gateway deba ser autenticada y verificada por medio de un token JWT. Tendremos un componente encargado de la generación de tokens y la validación del token para las autorizaciones, a su vez este servicio será llamado desde el API Gateway para interceptar cualquier petición entrante.

Razonamiento sobre las principales decisiones de arquitectura tomadas en este modelo

Cuando utilizamos tácticas de arquitectura como son la autenticación y autorización por medio de un servicio, el cual interceptara todas las peticiones entrantes a través del API Gateway, garantizamos que los servicios expuestos solo retornarán o harán comandos de acuerdo con los privilegios dados con anterioridad al momento de generar dicho token en la autenticación. Todas las peticiones entrantes deben ser atendidas por el API Gateway, garantizando así que el usuario o servicio que realice la petición sea realmente el servicio correcto. Las peticiones serán validadas por medio de un token JWT, el token JWT contendrá la información de la autorización de las peticiones entrantes, así mitigamos problemas de seguridad como son la suplantación o la elevación de privilegios.

Vista Funcional – (Estilo Microservicios)

Proyecto		**ID**	VC-001	**Elaboración**	Ronald Carrascal	**Versión**	1.0
Vista			**Modelo**	Funcional			

Tácticas de arquitectura utilizadas para favorecer la Escalabilidad y Mantenibilidad

Se dispondrá de estilo microservicios para favorecer la escalabilidad, permitiendo escalar horizontalmente levantando instancias de cada microservicios de manera independiente cuando fuese necesario. Esto nos ayudara a solo escalar los servicios que necesiten dicho escalamiento.

Los servicios estarán divididos teniendo en cuenta de que cada servicio se dedique solo a su dominio especifico permitiendo una alta cohesión y bajo acoplamiento, permitiendo asi que cada servicio este totalmente independiente, y en caso de extender el sistema baste con agregar solo el servicio de ser posible. Adicional a lo anterior se tomó la decisión de usar colas de mensajeria en los servicios de pagos y notificaciones, permitiendo así disminuir el acomplamiento entre los servicios de pagos y de reservas.

Razonamiento sobre las principales decisiones de arquitectura tomadas en este modelo

La escalabilidad es uno de los atributos mas importantes, y tambien de vital importancia solo escalar los servicios que necesitamos, Si los componentes están fuertemente acoplados, cualquier aumento de carga en un módulo afecta a todo el sistema. Separar responsabilidades permite escalar únicamente las partes que realmente lo necesitan.

Al tener los servicios separados estamos favoreciendo la facilidad de modificacion, ya que el agregar una nueva funcionalidad nos permitira solo extender nuestro sistema sin modificar los servivios existentes. Es importante siempre favorecer la extensibilidad y evitar al minimo la modificacion de los servicios existentes, para eso este modelo permite tener los servicios con un bajo acoplamiento y una alta cohesion. El uso de colas de mensajeria tambien nos permite tener un bajo acomplamiento de servicios, por ejemplo el servicio de pagos.

Vista Información

Proyecto		**ID**	VC-001	**Elaboración**	Ronald Carrascal	**Versión**	1.0
Vista			**Modelo**	Funcional			

Tácticas de arquitectura utilizadas para favorecer Rendimiento

En el servicio de busqueda de alojamiento se usará una base de datos en memoria como cache (Redis) para que las consultas se hagan sobre esta, adicional existira una cola de mensajerias que estara alimentando la informacion en redis por medio del servicio de inventario, de esta manera las consultas a los alojamientos estarán completamente en una memoria de facil y rapido acceso, tambien pudiendo escalar el servicio de busqueda de alojamiento. Toda la actualizacion de los alojamientos se realizará de manera asincrona por medio de una cola de mensajería CommandUpdate

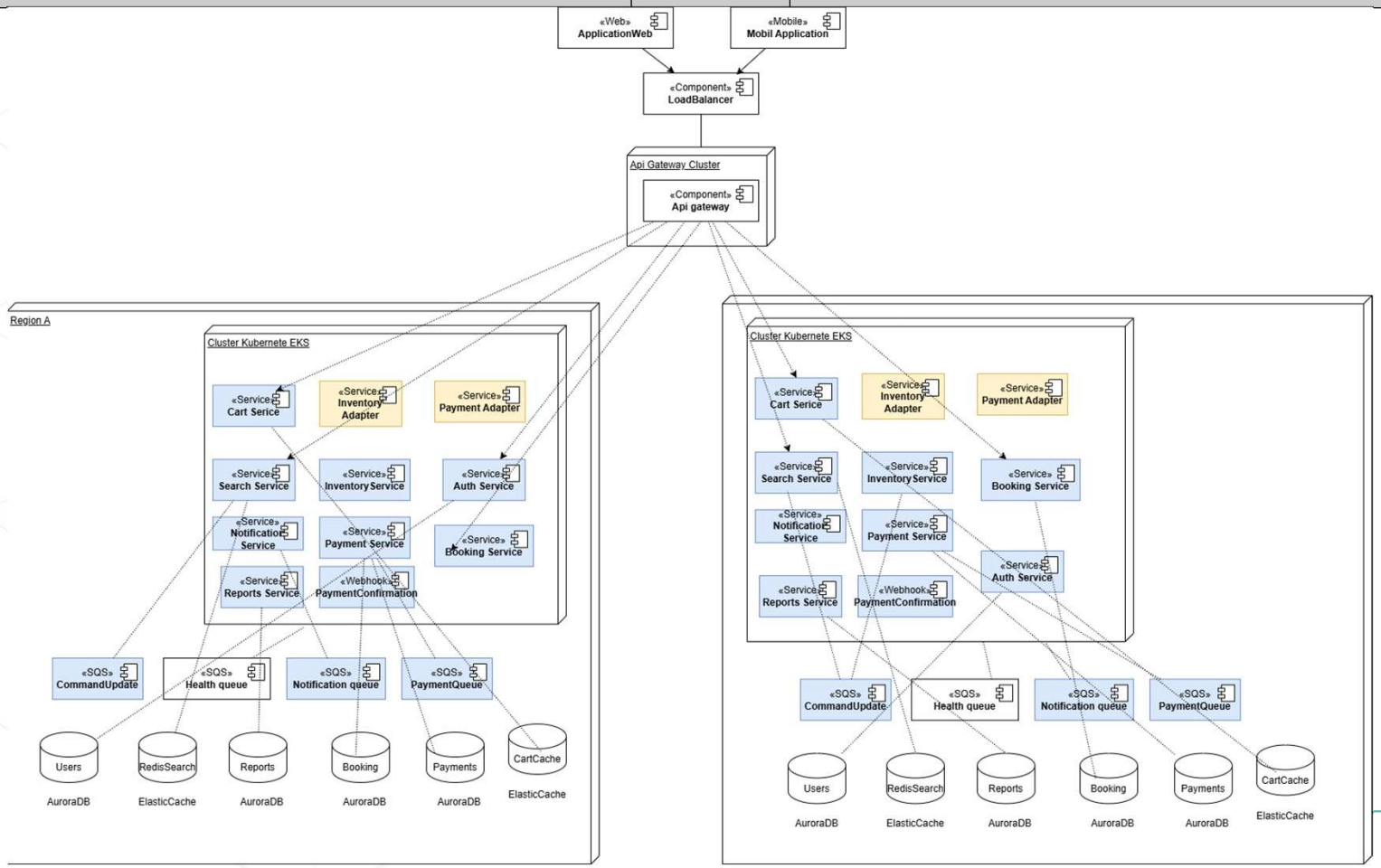
Razonamiento sobre las principales decisiones de arquitectura tomadas en este modelo

Tener un sistema de cache nos permite tener información del alojamiento en tiempo real con acceso mucho más facil que estar consultando el servicio de cada alojamiento de manera sincrona. La asincronia de la actualizacion nos permitira tener el servicio de búsqueda completamente actualizado favoreciendo la fiabilidad de los datos.

Vista de despliegue

Proyecto		ID	VC-005	Elaboración	Ronald Carrascal	Versión	1.2
Vista	Despliegue		Modelo	Despliegue			

Convención
UML



Vista de despliegue

Proyecto		ID	VC-005	Elaboración	Ronald Carrascal	Versión	1.1
Vista	Despliegue		Modelo	Despliegue			

Tácticas de arquitectura utilizadas para favorecer la disponibilidad y resiliencia

De acuerdo al estilo de arquitectura de microservicios, el diagrama especifica que cada componente se despliega en un nodo independiente. Esto permite escalar horizontalmente al desplegar múltiples instancias de cada nodo.

Además, se emplea la táctica de monitoreo para verificar el estado de salud de todas las instancias del sistema. Esta verificación se realiza de forma asincrónica a través del componente de colas. Para evitar sobrecargar el diagrama, la asociación de este componente de mensajería con el resto del sistema se muestra únicamente en algunos nodos.

Por último, ambas tácticas se complementan con la táctica de retiro de servicio, la cual, una vez identificado un nodo en estado de falla por medio del monitor, inicia un proceso de reparación.

Razonamiento sobre las principales decisiones de arquitectura tomadas en este modelo

Al contar con múltiples instancias para un mismo servicio (escalamiento horizontal), se garantiza que, en caso de falla de alguna instancia, las demás puedan asumir la carga total del servicio. Para lograr esto, es necesario detectar, mediante un servicio de monitoreo, cuándo una instancia se encuentra en estado de fallo, a fin de redistribuir su tráfico a las demás instancias operativas.

Adicionalmente, la comunicación entre algunos componentes del sistema es asincrónica a través de un componente de mensajería. Esto favorece el desacoplamiento de los servicios y permite que los comandos o eventos que no puedan ser procesados por una instancia sean manejados por otras. Incluso, si solo existía un único nodo activo para el componente, este podrá retomar el procesamiento de los mensajes recibidos mientras estuvo en proceso de reparación.

Vista de despliegue

Proyecto		ID	VC-005	Elaboración	Andrés Peña	Versión	1.0
Vista	Despliegue		Modelo	Despliegue			

Tácticas de arquitectura utilizadas para favorecer la seguridad

Se implementa la táctica de **Autenticación y Autorización** en el Api Gateway.

Razonamiento sobre las principales decisiones de arquitectura tomadas en este modelo

El sistema ya cuenta con un API Gateway, lo cual reduce la superficie de ataque al ofrecer un único punto de entrada a los actores y servicios externos a este. Por otra parte, existe un componente de seguridad (Auth Service) desplegado en su propio nodo, lo cual permite una mayor cohesión y disponibilidad.

Vista Despliegue

Proyecto		**ID**	VC-001	**Elaboración**	Ronald Carrascal	**Versión**	1.0
Vista			**Modelo**	Funcional			

Tácticas de arquitectura utilizadas para favorecer la Escalabilidad y Mantenibilidad

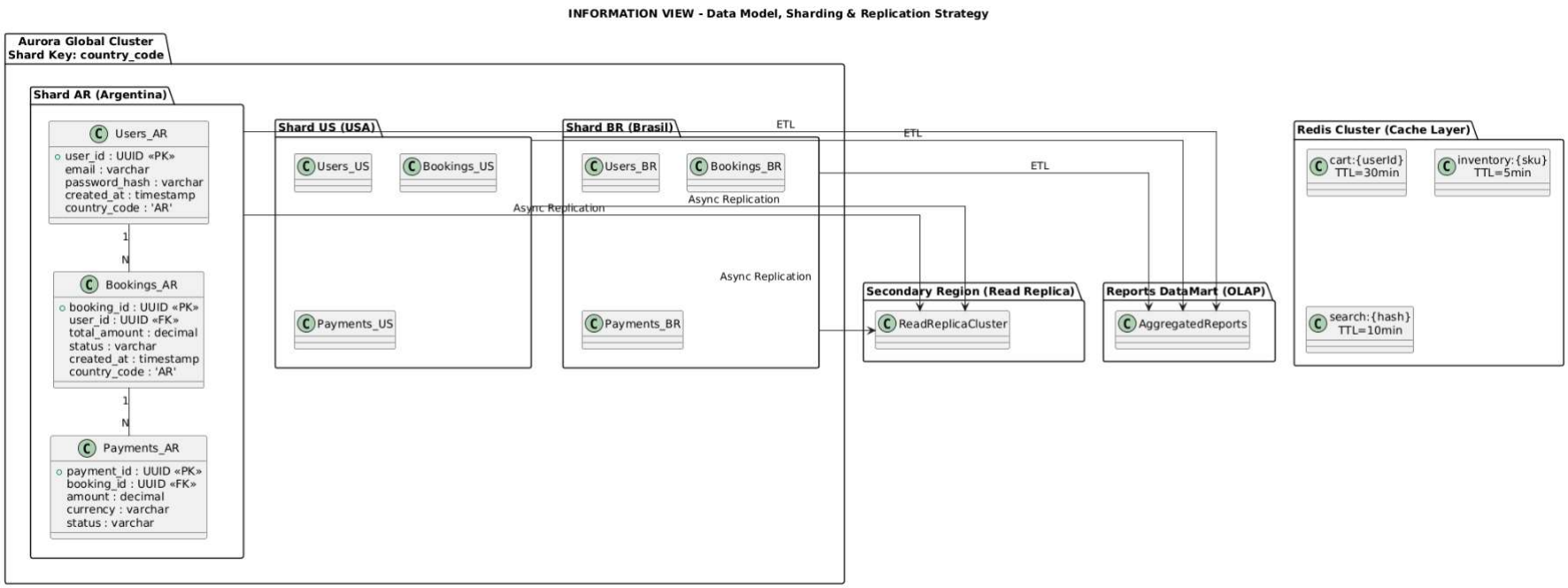
Los microservicios estarán desplegados en cluster kubernetes permitiendo escalar cada servicio o instancia de forma facil, adicional a eso si utilizamos servicios en la nube como AWS, podemos configurar el escalamiento de manera automática.

Razonamiento sobre las principales decisiones de arquitectura tomadas en este modelo

La utilización de kubernetes dentro de nuestro modelo nos permite automatizar de manera importante la creacion de instancias de cada servicio y orquestar cada contenedor de tal manera que podemos mantener separados de forma importante cada servicio.

Vista de Información

Proyecto		ID	VC-004	Elaboración	Fabrizio Cucina	Versión	1.0	Convención
Vista			Modelo	Información				



Vista de Información

Proyecto		**ID**	VC-004	**Elaboración**	Fabrizio Cucina	**Versión**	1.0
Vista			**Modelo**	Información			

Tácticas de arquitectura utilizadas para favorecer la disponibilidad y resiliencia

De acuerdo al estilo de arquitectura de microservicios, el diagrama especifica que cada componente se despliega de manera independiente y se puedes escalar servicios dependiendo de la demanda.

Además, se emplea la táctica de monitoreo para verificar el estado de salud de todas las instancias del sistema. Esta verificación se realiza de forma asincrónica a través del componente Queue. Este con el objetivo de mantener la disponibilidad de los servicios. ambas tácticas se complementan con la táctica de retiro de servicio, la cual removera un servicio en caso de fallo.

Razonamiento sobre las principales decisiones de arquitectura tomadas en este modelo

Cuando un servicio se escala horizontalmente con múltiples instancias, se asegura que, en caso de fallo de una de ellas, las demás puedan asumir la carga total. Para lograrlo, es fundamental contar con un servicio de monitoreo que detecte cuándo una instancia deja de estar operativa y redistribuya su tráfico a las instancias activas.
Además, algunos componentes del sistema se comunican de forma asincrónica a través de un sistema de mensajería. Esto permite un mayor desacoplamiento entre los servicios y garantiza que los comandos o eventos no procesados por una instancia puedan ser gestionados por otras.

Vista Información

Proyecto		**ID**	VC-001	**Elaboración**	Ronald Carrascal	**Versión**	1.0
Vista			**Modelo**	Funcional			

Tácticas de arquitectura utilizadas para favorecer Rendimiento

Toda la actualizacion de los alojamientos se realizará de manera asincrona por medio de una cola de mensajería CommandUpdate. Cada informacion del alojamiento estara disponible en una base de datos liviana como lo es Redis para que sea accedida con baja latencia desde el servicio de busqueda de alojamiento.

La sincronizacion permitira mantener un registro de ultima actualización permitiendo identificar cuando ha sido actualizado el registro, para evitar leer informacion no actualizada.

Razonamiento sobre las principales decisiones de arquitectura tomadas en este modelo

La informacion estara disponible con baja latencia si no tenemos que ir por cada peticion a consultar la informacion en los sistemas terceros de los hoteles, adicional a esto desacoplamos nuestro sistema de los sistemas de terceros.

© - Derechos Reservados: la presente obra, y en general todos sus contenidos, se encuentran protegidos por las normas internacionales y nacionales vigentes sobre propiedad Intelectual, por lo tanto su utilización parcial o total, reproducción, comunicación pública, transformación, distribución, alquiler, préstamo público e importación, total o parcial, en todo o en parte, en formato impreso o digital y en cualquier formato conocido o por conocer, se encuentran prohibidos, y solo serán lícitos en la medida en que se cuente con la autorización previa y expresa por escrito de la Universidad de los Andes.

De igual manera, la utilización de la imagen de las personas, docentes o estudiantes, sin su previa autorización está expresamente prohibida. En caso de incumplirse con lo mencionado, se procederá de conformidad con los reglamentos y políticas de la universidad, sin perjuicio de las demás acciones legales aplicables.